SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA0606 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:100**

Annexure A

Experiments

Code of Conduct:

I Prudhvi Raj.B(192324274) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Prudhvi raj.b

Annexure A

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a realworld dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

3. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

4. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

5. Design and conduct an experiment to implement a Decision Tree classifier on a realworld dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Naïve Bayes – Feature Sets + Laplace Smoothing

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.3, random_state=42, stratify=y
)

feature_sets = {
   "2 Features": [0, 1],
   "3 Features": [0, 1, 2],
   "4 Features": [0, 1, 2, 3]
}

for name, features in feature_sets.items():

   Xtr = X_train[:, features]
   Xte = X_test[:, features]

   model = GaussianNB()
   model.fit(Xtr, y_train)

   y_pred = model.predict(Xte)

   print("\n", name)
   print("Accuracy :", accuracy_score(y_test, y_pred))
   print("Precision:", precision_score(y_test, y_pred, average="weighted"))
   print("Recall   :", recall_score(y_test, y_pred, average="weighted"))

print("\nWith Laplace Smoothing:")

# Discretize continuous features
X_discrete = (X > X.mean(axis=0)).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
   X_discrete, y, test_size=0.3, random_state=42, stratify=y
)
```

```
from sklearn.naive_bayes import MultinomialNB

for alpha in [0, 1]:
    if alpha == 0:
        model = MultinomialNB(alpha=1e-10)
    else:
        model = MultinomialNB(alpha=alpha)

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print("\nAlpha =", alpha)
    print("Accuracy :", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred, average="weighted"))
    print("Recall   :", recall_score(y_test, y_pred, average="weighted"))
```

OUTPUT:

```
 2 Features
Accuracy : 0.6888888888888889
Precision: 0.6794871794871795
Recall   : 0.6888888888888889

 3 Features
Accuracy : 0.8888888888888888
Precision: 0.8898809523809523
Recall   : 0.8888888888888888

 4 Features
Accuracy : 0.9111111111111111
Precision: 0.9155354449472096
Recall   : 0.9111111111111111

With Laplace Smoothing:

Alpha = 0
Accuracy : 0.6222222222222222
Precision: 0.6122994652406417
Recall   : 0.6222222222222222

Alpha = 1
Accuracy : 0.6222222222222222
Precision: 0.6122994652406417
Recall   : 0.6222222222222222
```

2. MLE vs Bayesian Estimation

```
import numpy as np

np.random.seed(42)

data = np.random.normal(50, 10, 100)

sample_sizes = [10, 50, 100]

# Prior parameters
prior_mean = 50
prior_variance = 25

for n in sample_sizes:

  sample = data[:n]

  # MLE
  mle_mean = np.mean(sample)
  mle_variance = np.var(sample)

  # Bayesian estimation of mean
  sample_variance = np.var(sample, ddof=1)

  posterior_variance = 1 / (
    (1 / prior_variance) + (n / sample_variance)
  )

  posterior_mean = posterior_variance * (
    (prior_mean / prior_variance) +
    (n * mle_mean / sample_variance)
  )

  print("\nSample Size:", n)

  print("MLE Mean     :", mle_mean)
  print("MLE Variance  :", mle_variance)

  print("Bayesian Mean :", posterior_mean)
  print("Bayesian Variance:", posterior_variance)
```

OUTPUT:

```
Sample Size: 10
MLE Mean      : 54.48061111698756
MLE Variance  : 47.046694521315665
Bayesian Mean : 53.70575170228069
Bayesian Variance: 4.323402514051629

Sample Size: 50
MLE Mean      : 47.745260947438595
MLE Variance  : 85.43026463778315
Bayesian Mean : 47.892253091276714
Bayesian Variance: 1.629813255675169

Sample Size: 100
MLE Mean      : 48.96153482605907
MLE Variance  : 81.65221946938584
Bayesian Mean : 48.99470045791475
Bayesian Variance: 0.7984290828411162
```

3. Bayesian Belief Network

```
# Bayesian Belief Network
# Attendance -> Study Hours -> Exam Result
# Attendance -> Exam Result

prob_attendance = {
  "High": 0.6,
  "Low": 0.4
}

prob_study = {
  "High": {"High": 0.8, "Low": 0.2},
  "Low": {"High": 0.3, "Low": 0.7}
}

prob_result = {
  ("High", "High"): 0.95,
  ("High", "Low"): 0.70,
  ("Low", "High"): 0.75,
  ("Low", "Low"): 0.20
}

def calculate_result(evidence_attendance=None, evidence_study=None):

  probability = 0

  for attendance in ["High", "Low"]:
```

```
        if evidence_attendance is not None:
            if attendance != evidence_attendance:
                continue
            p_attendance = 1
        else:
            p_attendance = prob_attendance[attendance]

        for study in ["High", "Low"]:

            if evidence_study is not None:
                if study != evidence_study:
                    continue
                p_study = 1
            else:
                p_study = prob_study[attendance][study]

            probability += (
                p_attendance *
                p_study *
                prob_result[(attendance, study)]
            )

    return probability


print("P(Pass) =", calculate_result())

print("P(Pass | Attendance=High) =",
      calculate_result(evidence_attendance="High"))

print("P(Pass | Attendance=Low) =",
      calculate_result(evidence_attendance="Low"))

print("P(Pass | Study=High) =",
      calculate_result(evidence_study="High"))

print("P(Pass | Study=Low) =",
      calculate_result(evidence_study="Low"))
```

OUTPUT:

```
P(Pass) = 0.6859999999999999
P(Pass | Attendance=High) = 0.9
P(Pass | Attendance=Low) = 0.365
P(Pass | Study=High) = 0.87
P(Pass | Study=Low) = 0.5
```

4. Candidate Elimination – Hypothesis Space

```
import csv

data = [
  ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
  ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
  ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
  ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"],
  ["Rainy", "Cold", "Normal", "Weak", "Cool", "Change", "No"],
  ["Sunny", "Warm", "Normal", "Weak", "Warm", "Same", "Yes"]
]

concepts = [row[:-1] for row in data]
target = [row[-1] for row in data]

# Initial Specific Hypothesis
S = ["0"] * len(concepts[0])

# Initial General Hypothesis
G = [["?"] * len(S)]

for i in range(len(concepts)):

  x = concepts[i]

  if target[i] == "Yes":

    for j in range(len(S)):

      if S[j] == "0":
        S[j] = x[j]

      elif S[j] != x[j]:
        S[j] = "?"

    new_G = []

    for g in G:
```

```
        consistent = True

        for j in range(len(S)):
            if g[j] != "?" and g[j] != S[j]:
                consistent = False

        if consistent:
            new_G.append(g)

    G = new_G

else:

    new_G = []

    for g in G:

        for j in range(len(S)):

            if g[j] == "?" and S[j] != "?":
                h = g.copy()
                h[j] = S[j]

                if h not in new_G:
                    new_G.append(h)

    G = new_G

print("Specific Hypothesis:")
print(S)

print("\nGeneral Hypothesis:")
for g in G:
    print(g)

print("\nNumber of General Hypotheses:", len(G))
print("Number of Attributes:", len(S))
print("Number of Training Examples:", len(data))
```

OUTPUT:

```
Specific Hypothesis:
['Sunny', 'Warm', '?', '?', '?', '?']

General Hypothesis:
['Sunny', 'Warm', '?', '?', '?', '?']

Number of General Hypotheses: 1
Number of Attributes: 6
Number of Training Examples: 6
```

5. Decision Tree – Entropy vs Gini + Tree Depth

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.3, random_state=42, stratify=y
)

criteria = ["entropy", "gini"]
depths = [2, 3, 4, None]

for criterion in criteria:

   print("\nCriterion:", criterion)

   for depth in depths:

      model = DecisionTreeClassifier(
         criterion=criterion,
         max_depth=depth,
         random_state=42
      )

      model.fit(X_train, y_train)

      y_pred = model.predict(X_test)

      accuracy = accuracy_score(y_test, y_pred)
      f1 = f1_score(y_test, y_pred, average="weighted")
```

```
    print(
        "Depth:", depth,
        " Accuracy:", round(accuracy, 4),
        " F1-Score:", round(f1, 4)
    )
```

OUTPUT:

```
Criterion: entropy
Depth: 2  Accuracy: 0.8889  F1-Score: 0.8888
Depth: 3  Accuracy: 0.9333  F1-Score: 0.9327
Depth: 4  Accuracy: 0.8889  F1-Score: 0.8888
Depth: None  Accuracy: 0.8889  F1-Score: 0.8888

Criterion: gini
Depth: 2  Accuracy: 0.8889  F1-Score: 0.8888
Depth: 3  Accuracy: 0.9778  F1-Score: 0.9778
Depth: 4  Accuracy: 0.8889  F1-Score: 0.8888
Depth: None  Accuracy: 0.9333  F1-Score: 0.9327
```